PROJECT REVIEW 2

Cloud Application Performance Monitoring and Alerting System

Literature Survey • Research Gaps • Proposed Approach

Team Members
10692373319 Shaik Athar
10692373321 Shaik Moshin
10692373333 Shaik Mukarram
10692373322 Shaik Zahed

Guided by: Mr. S. Naveen

Research-based review of cloud performance monitoring and alerting

Project Overview and Research Problem

Project focus

Continuously observe cloud applications, detect performance degradation, and notify the responsible team.

Signals monitored

Response time, throughput, error rate, CPU, memory, availability, network and storage I/O.

Core workflow

Cloud services → collectors → metrics/logs/traces → analysis and alert rules → dashboard and notifications.

Research problem

- Cloud workloads and service dependencies change dynamically.
- Large volumes of metrics, logs and traces make diagnosis difficult.
- Fixed thresholds can create noisy alerts or miss unusual behaviour.
- Monitoring itself can add overhead and operational cost.

Review objective

- Study existing cloud monitoring and anomaly-detection methods.
- Compare the data used, detection approach and limitations.
- Identify a practical gap for an integrated monitoring and alerting prototype.

Literature Survey — Papers 1 to 4

1. Roots — Jayathilaka et al. (2020)

Full-stack PaaS monitoring; collects request, runtime and service metrics. Detects anomalies online and helps locate bottlenecks.

Limitation for our scope: designed around a PaaS platform and its internal components.

2. Farshchi et al. (2018)

Correlates cloud-operation logs with resource metrics; selects relevant metrics and derives runtime assertions for anomaly detection.

Limitation: focuses on operation/upgrade scenarios and metric selection.

3. TraceWeaver — Ashok et al. (2024)

Reconstructs microservice request traces using timestamps and call-graph information without changing application code.

Limitation: focuses on reconstructing traces, not a complete metrics-to-alerting workflow.

4. DeCaf — Bansal et al. (2020)

Uses machine learning and pattern mining on service logs to diagnose and triage KPI issues in large cloud services.

Limitation: relies on service logs and historical operational data.

Literature Survey — Papers 5 to 8

5. ServiceAnomaly (2024)

Combines distributed traces with profiling metrics to model normal microservice behaviour and detect anomalies.

Limitation: depends on trace and profiling data being available.

6. TracePicker (2025)

Uses optimization-based trace sampling to reduce storage and analysis burden while retaining useful traces.

Limitation: addresses trace selection rather than end-to-end alert management.

7. Nõu et al. (2025)

Measures overhead from distributed tracing frameworks; reports that tracing can affect throughput and latency.

Limitation: evaluates tracing overhead rather than proposing a full monitoring platform.

8. Hrusto et al. (2026)

Maps 104 studies on monitoring data and anomaly detection in cloud-based systems; highlights data and preprocessing challenges.

Limitation: a systematic mapping study, not a deployed monitoring solution.

Literature Survey Comparison

Study	Main input / method	Main contribution	Limitation / scope
Roots (2020)	Requests, runtime and PaaS metrics	Online anomaly detection and bottleneck analysis	PaaS-oriented full-stack instrumentation
Farshchi et al. (2018)	Operation logs + resource metrics	Metric selection and runtime assertions	Specific cloud-operation scenarios
TraceWeaver (2024)	Timestamps + call graphs	Trace reconstruction without app changes	Trace reconstruction, not full alert workflow
DeCaf (2020)	Service logs + ML/pattern mining	KPI diagnosis and triage	Log-focused; depends on historical data
ServiceAnomaly (2024)	Distributed traces + profiling metrics	Combined-signal microservice anomaly detection	Needs trace/profiling data
TracePicker (2025)	Distributed traces	Reduces trace sampling/storage burden	Trace sampling focus
Nõu et al. (2025)	OpenTelemetry / Elastic APM traces	Quantifies tracing performance overhead	Overhead evaluation focus
Hrusto et al. (2026)	Review of 104 studies	Maps data, preprocessing and detection methods	Survey, not an implemented system

Research Gaps Identified

1. Fragmented observability

Metrics, logs and traces are often handled by separate tools, making a unified view difficult.

2. Noisy threshold alerts

Static thresholds may trigger too often during normal workload variation or fail to capture context.

3. Diagnosis takes time

A metric alert may show that a service is unhealthy but not clearly explain which dependency caused it.

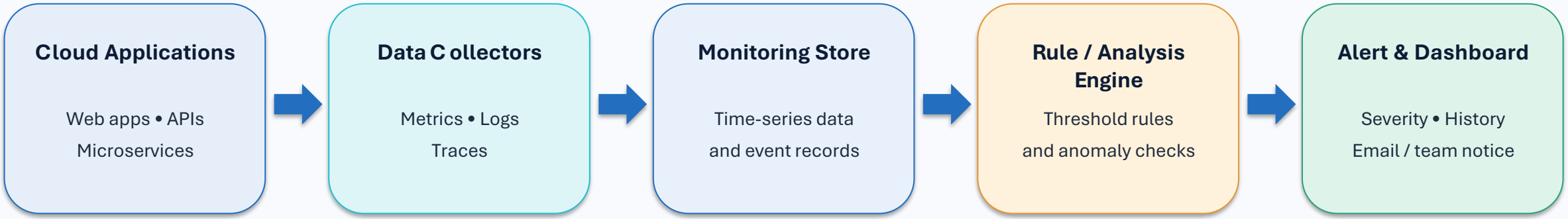
4. Monitoring overhead

Collecting and exporting large trace volumes can consume resources and affect application performance.

Research gap for this project

A lightweight, integrated prototype that brings key metrics, logs and traces into one dashboard, applies configurable alert rules, and provides useful context for investigating an alert.

Proposed System Architecture



Example alert rule

Condition

CPU utilization > 80% for 5 minutes

Severity

Warning / Critical based on rule configuration

Action

Record event and notify the responsible team

Proposed design follows the project's Review 1 architecture and adds research-informed considerations for noise and overhead.

How the Proposed Project Differs

Existing research approaches

- Some systems focus on full-stack monitoring within a particular cloud platform.
- Some focus on log-based diagnosis, trace reconstruction, or trace sampling.
- Some research studies anomaly detection methods or the overhead of tracing.
- These works solve important parts of the monitoring problem.

Proposed student project

- Combines key metrics, logs and traces in one monitoring workflow.
- Uses configurable thresholds and severity-based alerts.
- Presents dashboard information and alert history in one place.
- Targets a simple, understandable prototype suitable for a small cloud application.

Planned evaluation

Measure detection time, alert count/noise, dashboard visibility and monitoring overhead under normal and simulated load conditions.

Expected Outcomes and Evaluation Plan

Unified visibility

A single dashboard for service health, response time, errors and resource usage.

Timely alerts

Configurable warning and critical rules with recorded alert history.

Faster investigation

Related metric and event context helps the user begin diagnosing a service issue.

Evaluation plan (to be carried out during implementation)

- Run the application under normal load and controlled high-load/error scenarios.
- Check whether configured conditions trigger the correct alert and severity.
- Record detection delay, number of false/noisy alerts, and resource overhead.
- Compare the dashboard's visibility and investigation workflow with separate/manual monitoring.

Note: These are proposed evaluation activities; results should be added only after testing.

References (IEEE Style)

[1] H. Jayathilaka, C. Krintz, and R. Wolski, “Detecting Performance Anomalies in Cloud Platform Applications,” IEEE Trans. Cloud Comput., vol. 8, no. 3, pp. 764–777, 2020, doi: 10.1109/TCC.2018.2808289.

[2] M. Farshchi, J. G. Schneider, I. Weber, and J. Grundy, “Metric selection and anomaly detection for cloud operations using log and metric correlation analysis,” J. Syst. Softw., vol. 137, pp. 531–549, 2018, doi: 10.1016/j.jss.2017.03.012.

[3] S. Ashok, V. Harsh, B. Godfrey, R. Mittal, S. Parthasarathy, and L. Shwartz, “TraceWeaver: Distributed Request Tracing for Microservices Without Application Modification,” in Proc. ACM SIGCOMM, 2024.

[4] C. Bansal, S. Renganathan, A. Asudani, O. Midy, and M. Janakiraman, “DeCaf: Diagnosing and Triaging Performance Issues in Large-Scale Cloud Services,” in Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE), 2020.

[5] “ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics,” J. Syst. Softw., vol. 209, Art. no. 111917, 2024, doi: 10.1016/j.jss.2023.111917.

[6] S. Xie, J. Wang, M. Li, P. Chen, J. Xuan, and B. Li, “TracePicker: Optimization-Based Trace Sampling for Microservice-Based Systems,” Proc. ACM Softw. Eng., vol. 2, FSE, Art. no. FSE081, pp. 1802–1823, 2025, doi: 10.1145/3729351.

[7] A. Nõu, S. Talluri, A. Iosup, and D. Bonetta, “Investigating Performance Overhead of Distributed Tracing in Microservices and Serverless Systems,” in Proc. ICPE ’25 Companion, pp. 162–166, 2025, doi: 10.1145/3680256.3721316.

[8] A. Hrusto, N. B. Ali, E. Engström, and Y. Wang, “Monitoring Data for Anomaly Detection in Cloud-Based Systems: A Systematic Mapping Study,” ACM Trans. Softw. Eng. Methodol., vol. 35, no. 4, Art. no. 91, 2026, doi: 10.1145/3744556.